

Verification using UVM : Introduction

Pravin Zode

Outline

- Basic of Design Verification
- Why Methodology ?
- Evolution of UVM
- Comparison of SV and UVM
- Key features of UVM
- Learning Resources

Why Design matters for Verification ???

- For a verification engineer, you must understand how designs are built. Why ???
- You can predict where bugs are likely to occur
- You can write meaningful test cases
- You can debug waveforms faster
- You speak the same language as design engineers

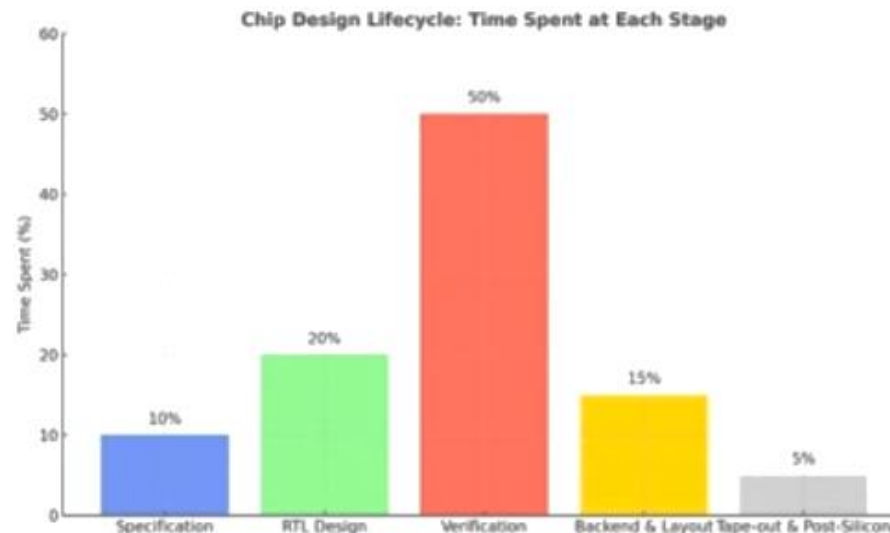
Design Verification

- Ensures functional correctness
- Reduces cost of Failure
- Takes more than the design
- Essential for first-time silicon success

Chip Design

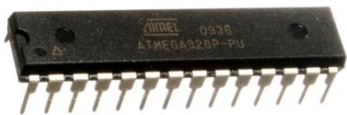
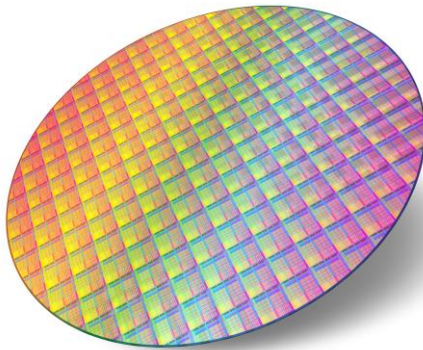
Is Not only Writing RTL Code

**But also making sure that
Works perfectly**

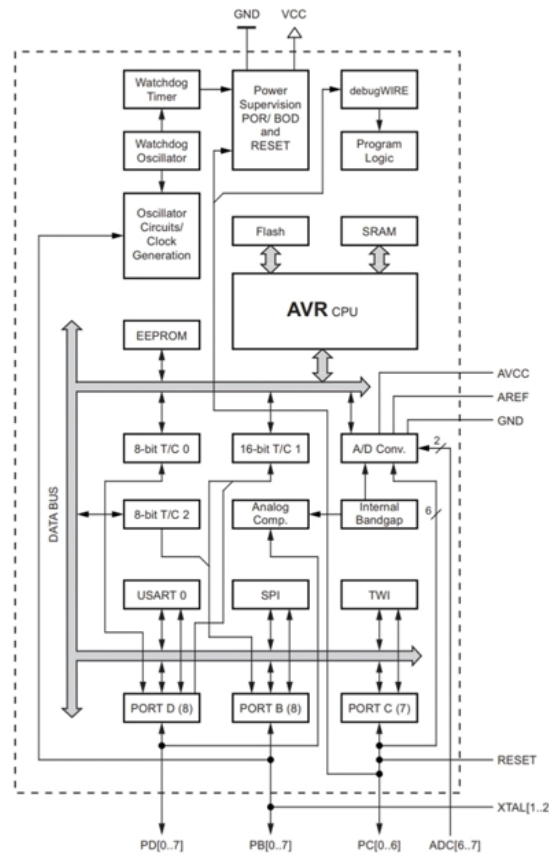


Verification

Post-Silicon Verification



System Level Verification



Pre-Silicon Verification

```
module cfs_synch_fifo #(
    parameter DATA_WIDTH = 32,
    parameter FIFO_DEPTH = 8,
) (
    input                reset_n,

    input                push_clk,
    input                push_valid,
    input[DATA_WIDTH-1:0] push_data,
    output wire          push_ready,

    //Full flag - in PUSH clock domain
    output reg          push_full,

    //Empty flag - in PUSH clock domain
    output reg          push_empty,

    //FIFO level - in PUSH clock domain
    output reg[CNT_WIDTH:0] push_fifo_lv1,

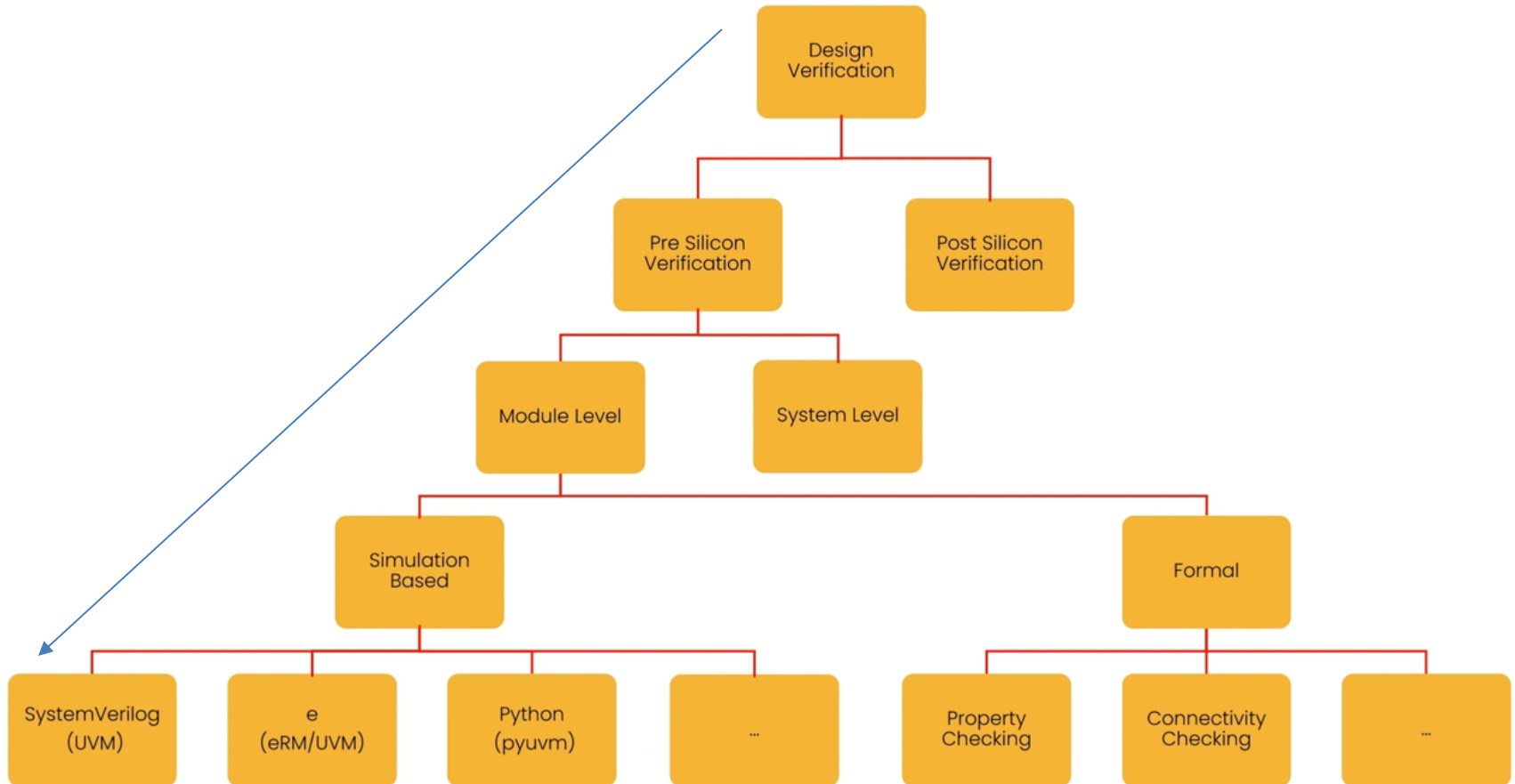
    input                pop_clk,
    output wire          pop_valid,
    output wire[DATA_WIDTH-1:0] pop_data,
    input                pop_ready,

    //Full flag - in POP clock domain
    output reg          pop_full,

    //Empty flag - in POP clock domain
    output reg          pop_empty,

    //FIFO level - in POP clock domain
    output reg[CNT_WIDTH:0] pop_fifo_lv1
);
--
endmodule
```

Verification



Why UVM ???

- SystemVerilog is a powerful hardware description language that combines OOP and advanced verification features
- SV provides a robust environment for verification
- It lacks standardized methodology for structuring testbenches in scalable and reusable manner
- This is where Universal Verification Methodology (UVM) comes into play
- UVM is framework on top of SystemVerilog making it modular, reusable and scalable for complex designs

Early Days (Verilog and Directed Testing 1990s)

- Initially, verification was done using Verilog testbenches with direct stimulus
- Testbenches were simple procedural models that applied input vectors and checked outputs

Issues

- Writing directed test cases manually was time-consuming
- Hard to scale for complex designs
- Low reusability: Every project required a new testbench

SystemVerilog(2005, IEEE 1800-2005 Standard)

- SystemVerilog introduced OOP concepts like classes, polymorphism, and randomization, making testbenches more structured

Key improvements over Verilog

- **Constrained random stimulus generation** to increase test coverage
- **Functional coverage** to track what scenarios are tested
- **Assertions** for checking design properties

Limitations of SystemVerilog

- No standard structure for testbenches
- Every team had to design their own verification framework from scratch
- No guidelines for stimulus generation, checking, and coverage collection
- Reusability across projects was limited

Problems with SystemVerilog TB

Problems	What Happens
Poor reusability	Code rewrite for every project
No standard Structure	Every engineer writes differently
Configuration chaos	Hardcoded values everywhere
Scaling Issues	Testbench becomes unmanageable
Debug difficulty	Hard to trace flow

Why Methodology

- HVL Limitations : SystemVerilog, Vera, e etc...
- Reusable and portability : Plug and Play not possible
- Develop testcases independent of testbench environment
- To be able to change the objects from the top level
- Does not have configuration facility for the TB architecture

What is Methodology ?

- Best practices by Verification experts
 - Standard framework to build the verification environment
- Base class library
 - For ex. uvm_component, uvm_sequence_item, uvm_object, etc..
- Standard communication mechanism to achieve interoperability
 - TLM 1.0 2.0 ports, exports etc....
- Configuration of the testbench from the top level
 - Factory, configuration methods etc...
- Generate scenarios independent of testbench environment
 - Sequences , sequence items, etc...

Evolution of UVM

- Major EDA vendors had their own methodologies
 - Mentor Graphics : AVM for SystemVerilog
 - Cadence : eRM for Specman, e, uRM for SystemVerilog
 - Synopsys – VMM for SystemVerilog
- Open Verification Methodology (OVM)
 - Joint development initiative by Mentor Graphics and Cadence
 - Open , Interoperable, SystemVerilog based Methodology
- Universal Verification Methodology (UVM)
 - Common methodology by Mentor Graphics, Cadence and Synopsys
 - Interoperability – Multiple language support
 - Based on OVM proven library
 - Scales from Block level to System level

Universal Verification Methodology (UVM)

- A methodology is a **systematic approach** or **framework** that provides **guidelines, strategies**, and **best practices** for accomplishing a task
- UVM is methodology built on top of SystemVerilog
- It is framework or library that provides a structured way to organize verification environments using SystemVerilog constructs
- UVM helps in managing the complexity of testbenches by introducing standardized components, phases, and best practices , making it easier to verify large and complex designs

UVM 2010 (IEEE 1800.2 Standard)

- UVM was introduced as a standardized, industry-wide methodology, combining the best features of OVM and VMM
- Developed by Accelera, it became an IEEE standard (IEEE 1800.2) in 2017
- Now, all major vendors support UVM, ensuring compatibility across different tools

Comparision SV and UVM

Feature	SystemVerilog	UVM (Universal Verification Methodology)
Type	Hardware Description and Verification Language (HDVL)	Verification methodology (a library/framework) written in SystemVerilog
What It Is	A language standard used to describe hardware and write testbenches	A pre-defined structure and set of base classes (library) built on SystemVerilog
Defined By	IEEE 1800 Standard (SystemVerilog)	Developed by Accellera (UVM 1.2 is the latest non-IEEE version)
Used For	Writing RTL, assertions, basic and advanced testbenches	Writing modular, scalable, object-oriented testbenches with reusable components
Contains	Language features: logic, module, interface, class, covergroup, constraint	Class libraries: uvm_test, uvm_env, uvm_agent, uvm_monitor, uvm_sequence etc.

Key features of UVM

- **Standardized:** IEEE 1800.2 standard ensures interoperability across EDA tools and companies
- **Modular:** Pre-built base classes (drivers, monitors, scoreboards) accelerate development
- **Reusable:** Verification IP (VIP) can be shared across projects and organizations
- **Configurable:** Factory patterns and config_db enable flexible testbench adaptation
- **Industry-Adopted:** Supported by all major EDA vendors (Synopsys, Cadence, Mentor, Aldec)

What UVM Provides ?

- Testbench Components
- Transactions
- Phases
- Reporting
- Configuration
- Coverage
- RAL(Register Abstraction Level)

Learning Resources

- Accellera – Universal Verification Methodology (UVM) 1.1 User's Guide
- Mentor Graphics –UVM/OVM Documentation Verification Methodology online Cookbook
- IEEE_UVM_REF_Manual 1800.2-2017
- www.verificationacademy.com
- www.vlsiverify.com
- www.chipverify.com



**Thank you !
Happy Learning**